Chapter 9
Application Development Libraries

Abstract In this chapter we present several useful libraries and API which we could not categorize with any of the other libraries we presented in this part of the book. We first present the RPC (remote procedure call) library for remote invocation of predefined functions. We present a real-life example motivated from physical chemistry to use RPC for optimizing runtime using remote servers. In any discussion of cluster computing, data endianess issues always crop up, and a part of the RPC library is XDR (extensible data representation). We continue to present APIs for checksum, and hash signature computation to check and prevent transmission errors. An alternative to XDR is to use XML files, and library APIs for XML processing is presented in this chapter. For persistent storage we show examples of using Berkeley DB, and present examples using C++. A network based caching library (Memcache) is presented with examples in C++ and Python. To seamlessly use Python and other interactive languages, SWIG (the Simplified Wrapper Interface Generator) is presented.

Contents

9.1 RPC (remote procedure call) library 165
9.2 Checksum computation 173
9.3 OpenSSL 177
9.4 XML Processing 178
9.5 Berkeley DB 181
9.6 Memcached Library 185
9.7 SWIG interface generator 186
9.8 Conclusion 189

9.1 RPC (remote procedure call) library

The remote procedure call (RPC) library provides an API and abstraction to invoke pre-defined and pre-registered functions on remote servers. The library supports

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_9, © Springer Science+Business Media, LLC 2011

messaging, data-representation, client and server programming. There is a RPC protocol compiler *rpcgen* which converts a high level description of a function in RPC Language (a language similar to C), and generates C language code for client and server. The server needs to be registered with the `rpcbind` RPC server running on the server, by adding its service number into the `/etc/rpc` file.

We present a small example inspired from computational biology. Given a molecule with N atoms, and spectroscopic data containing M distances between the atoms, the goal of the *PDF Solver* is to assign x, y, z coordinates to the atoms which best matches the distances. We use a greedy minimization algorithm to solve for x, y, we assume z is fixed. The molecule data type is a simple collection of x, y, d vector of double. An example RPC Language file is shown below in Listing 9.1.

```
/*
 * RPC file for molecule computation
 * Sandeep Koranne, (C) 2010
 */
const MAX_MOLECULE_SIZE = 64;
const MAX_NUM_DISTANCES = 4096;

struct molecule {
       int N;
       int M;
       double coordinate_x< MAX_MOLECULE_SIZE >;
       double coordinate_y< MAX_MOLECULE_SIZE >;
       double distances< MAX_NUM_DISTANCES >;
};

typedef struct molecule* MPTR;

program MOLECULEPROG {
  version MOLVERS {
  MPTR  SOLVEPDF(MPTR) = 1;
  }
  = 1;
} = 0x20000002;
```

Listing 9.1 RPC Language description of molecule

We can generate the client and server code by running *rpcgen* as follows:

```
$rpcgen molecule.x
```

This generates the files:

1. molecule_clnt.c: the client functionality of `pdfsolver`,
2. molecule_svc.c: the server functionality of `pdfsolver`,
3. molecule_xdr.c: the XDR representation of molecule. See Section 9.1.1 for more details on XDR library.

In addition to these code files, we need to write additional code to actually perform the computation, and to call the `client` code which connects to the service running on the server. These are:

```
/* Molecular PDF Processing using RPC
 * (C) Sandeep Koranne, 2010
 */
#include <stdlib.h>
#include "molecule.h"
```

```
#include "molecule_ds.h"

extern  MPTR * solvepdf_1(MPTR *, CLIENT *);
extern  MPTR * solvepdf_1_svc(MPTR *, struct svc_req *);

static double ComputeDistanceEnergy( MPTR M ) {
  int i;
  double energy = 0.0;
  for( i=0; i < M->M; ++i ) {
    energy += M->distances.distances_val[i];
  }
  return energy;
}

static double ComputePointEnergy( int N,
          const double* X,
          const double* Y)
{
  int i,j;
  double energy = 0.0;
  for( i=0; i < N; ++i )
    for( j=0; j < N; ++j ) {
      double dx = X[i] - X[j];
      double dy = Y[i] - Y[j];
      if( ( X[i] == 100.0 ) || ( X[j] == 100.0 ) ) dx += 10000;
      if( ( Y[i] == 100.0 ) || ( Y[j] == 100.0 ) ) dy += 10000;
      energy += ( dx * dx ) + ( dy * dy );
    }

  return energy;
}


static double ComputeEnergy( MPTR M ) {
  int i;
  double energy = 0.0;
  const double *X = M->coordinate_x.coordinate_x_val;
  const double *Y = M->coordinate_y.coordinate_y_val;
  energy = ComputePointEnergy( M->N, X, Y );
  return energy;
}

/*
 * \function ProcessMoleculeForPDF
 * \description memory has been allocated for output
 */
static void ProcessMoleculeForPDF( MPTR input, MPTR output ) {
  const static unsigned int LOOP_COUNT = 1000;
  int i,j;
  int N,count;
  const double *iX = input->coordinate_x.coordinate_x_val;
  const double *iY = input->coordinate_y.coordinate_y_val;
  const double *iD = input->distances.distances_val;

  double *oX = output->coordinate_x.coordinate_x_val;
  double *oY = output->coordinate_y.coordinate_y_val;
  double *oD = output->distances.distances_val;

  double *X = malloc( input->N * sizeof( double ) );
  double *Y = malloc( input->N * sizeof( double ) );
  double d_energy = ComputeDistanceEnergy( input );
  double energy, min_energy = 1e10;
  N = input->N;

  for( i=0; i < input->N; ++i ) {
    X[i] = oX[i] = iX[i];
    Y[i] = oY[i] = iY[i];
```

```
  }
  for( i=0; i < input->N; ++i ) {
    oD[i] = iD[i];
  }
  /* for every X,Y which is 0, we have to solve */
  energy = ComputeEnergy( output ) - d_energy;
  for( i=0; i < LOOP_COUNT; ++i ) {
    double cur_energy = 0.0;
    /* set unsolved to random solutions */
    for( j=0; j < N; ++j ) {
      double prev_X;
      if( iX[j] != 100.0 ) continue;
      prev_X = X[j];      X[j] = drand48();
      /* now compute new energy */
      cur_energy = ComputePointEnergy( N, X, Y );
      if( cur_energy < energy )
  energy = cur_energy;
      else
  X[j] = prev_X;
    }
    /* Now do the Y loop */
    for( j=0; j < N; ++j ) {
      double prev_Y;
      if( iY[j] != 100.0 ) continue;
      prev_Y = Y[j];      Y[j] = drand48();
      /* now compute new energy */
      cur_energy = ComputePointEnergy( N, X, Y );
      if( cur_energy < energy )
  energy = cur_energy;
      else
  Y[j] = prev_Y;
    }
    /* printf("\n E %d  = %f ", i, energy ); */
  }
  count=0;
  for( i=0; i < N; ++i ) {
    oX[i] = X[i]; oY[i] = Y[i];
  }
  for( i=0; i < N; ++i )
    for( j=0; j < N; ++j ) {
      double dx = oX[i] - oX[j];
      double dy = oY[i] - oY[j];
      oD[ count++] = ( dx * dx ) + ( dy * dy );
    }
  free( X );
  free( Y );
}

MPTR* solvepdf_1( MPTR* input, CLIENT* client ) {
  static MPTR retval = NULL;
  MPTR actual_input;
  //if( retval ) { DestroyMolecule( retval ); retval = NULL; }
  actual_input = *input;
  retval = ConstructMolecule( actual_input->N, actual_input->M );
  ProcessMoleculeForPDF( actual_input, retval );
  return &retval;
}

MPTR* solvepdf_1_svc( MPTR* input, struct svc_req* req ) {
  CLIENT *client;
  printf("\n Memory used = ");
  system("cat /proc/self/statm");
  printf("\n");
  return( solvepdf_1( input, client ) );
}
```

Listing 9.2 Molecule processing service

The client code has to be wrapped as:

```
/* \file rmolecule.c
   \author Sandeep Koranne, (C) 2010
   \description Remote molecule processing
*/
#include <stdio.h>
#include "molecule.h"
#include "molecule_ds.h"

int main( int argc, char *argv [] ) {

  CLIENT *clnt;
  MPTR *result;
  MPTR input;
  char *server, *mfile;
  FILE *fp;
  server = argv[1];
  mfile =  argv[2];
  fp = fopen( mfile, "rt" );

  input = ReadMolecule( fp );
  InitializeMolecule( input );
  fclose( fp );

  clnt = clnt_create( server, MOLECULEPROG,
          MOLVERS, "udp" );
  if( clnt == NULL ) {
    clnt_pcreateerror(server);
    return 1;
  }
  result = solvepdf_1( &input, clnt );
  if( result == NULL ) {
    clnt_perror( clnt, server );
    return 1;
  }
  PrintMolecule( input );
  printf("\nMessage sent to server");
  clnt_destroy( clnt );
  /* now print the returned molecule */
  printf("\n Returned molcule = \n");
  PrintMolecule( *result );
  printf("\n");
  return 0;
}
```

Listing 9.3 Client for molecule processing service

We add the new service to `/etc/rpc`:

```
molecule_server 536870914
```

9.1.1 XDR : External Data Representation Library

XDR's approach to standardizing data representations is canonical. For example XDR defines a single byte order (big-endian) and uses the IEEE format for representing floating point numbers. Any program running on any machine can use XDR to create portable data by translating its local representation to the XDR standard

representations. The single standard completely de-couples programs that create or send portable data from those that use or receive portable data. Consider the program below:

```
/* \file rpc_writer.c
 * \author Sandeep Koranne, (C) 2010
 * \description Using XDR for serialization
 */

#include <stdio.h>
#include <rpc/types.h>
#include <rpc/xdr.h>

int main(int argc, char *argv[] ) {
  XDR xdrs;
  long i;
  xdrstdio_create( &xdrs, stdout, XDR_ENCODE );
  for( i = 0; i < 8; i++) {
    if( !xdr_long( &xdrs, &i ) ) {
      fprintf( stderr, "ERROR: xdr_long" );
      exit( 1 );
    }
  }
  return( 0 );
}
```

Listing 9.4 XDR writer example

and

```
/* \file rpc_reader.c
 * \author Sandeep Koranne, (C) 2010
 * \description Using XDR for serialization
 */

#include <stdio.h>
#include <rpc/types.h>
#include <rpc/xdr.h>

int main(int argc, char *argv[] ) {
  XDR xdrs;
  long i,j;
  xdrstdio_create( &xdrs, stdin, XDR_DECODE );
  for( i = 0; i < 8; i++) {
    if( !xdr_long( &xdrs, &j ) ) {
      fprintf( stderr, "ERROR: xdr_long" );
      exit( 1 );
    } else {
      printf("%d = %ld", i, j);
    }
  }
  return( 0 );
}
```

Listing 9.5 XDR reader example

The major XDR primitives are shown in Table 9.1.

At this time it is instructive to inspect the XDR description generated by `rpcgen` for a complex structure. Above in Section 9.1 we had discussed the RPC library with the example of computational biology using the `molecule` structure. The generated XDR is given below:

```
struct molecule {
```

Table 9.1 XDR functions for writing primitives

Directive	Description
`xdr_bool`	Boolean
`xdr_chars`	Char
`xdr_u_chars`	Unsigned char
`xdr_int`	Integer
`xdr_u_int`	Unsigned integer
`xdr_long`	Long
`xdr_u_long`	Unsigned long
`xdr_float`	Single precision floating point
`xdr_double`	Double precision floating point
`xdr_void`	void
`xdr_string`	null terminated byte sequence
`xdr_bytes`	byte sequence with length
`xdr_arrays`	arrays of arbitrary elements
`xdr_union`	Discriminated union
`xdr_reference`	Pointers
`xdrstdio_create`	initialize XDR stream
`xdrmem_create`	create XDR mem stream
`xdrrec_create`	create TCP/IP stream

```
  int N;
  int M;
  struct {
    u_int coordinate_x_len;
    double *coordinate_x_val;
  } coordinate_x;
  struct {
    u_int coordinate_y_len;
    double *coordinate_y_val;
  } coordinate_y;
  struct {
    u_int distances_len;
    double *distances_val;
  } distances;
};
typedef struct molecule molecule;

typedef molecule *MPTR;
```

Listing 9.6 Generated XDR for molecule.x

It can be seen that the arrays have been augmented with the length variable.

Another example with user defined XDR functions for writing structures is shown below:

```
/* \file system_load.h
 * \author Sandeep Koranne, (C) 2010
 * \description Using XDR for serialization
 */
#ifndef _system_load_h
#define _system_load_h
```

```
#include <rpc/types.h>
#include <rpc/xdr.h>

struct SystemLoad {
  char* system_name;
  float avg1, avg5, avg15;
};

extern bool_t SystemLoad_XDR( XDR *xdrs, struct SystemLoad* S );
#endif
```

Listing 9.7 XDR file for system load structure

```
/* XDR utility for own structure */
#include "system_load.h"

bool_t SystemLoad_XDR( XDR *xdrs, struct SystemLoad* S ) {
  return ( xdr_string( xdrs, &S->system_name, 64 ) &&
     xdr_float ( xdrs, &S->avg1 ) &&
     xdr_float ( xdrs, &S->avg5 ) &&
     xdr_float ( xdrs, &S->avg15 ) );
}
```

Listing 9.8 XDR file system load

```
/* \file rpc_info_writer.c
 * \author Sandeep Koranne, (C) 2010
 * \description Using XDR for serialization
 */

#include <stdio.h>
#include "system_load.h"

int main(int argc, char *argv[] ) {
  XDR xdrs;
  struct SystemLoad S;
  S.system_name = "celex";
  S.avg1 = 1.38;
  S.avg5 = 1.30;
  S.avg15 = 0.78;
  xdrstdio_create( &xdrs, stdout, XDR_ENCODE );
  SystemLoad_XDR( &xdrs, &S );
  return( 0 );
}
```

Listing 9.9 Server RPC file for system load

```
/* \file rpc_info_reader.c
 * \author Sandeep Koranne, (C) 2010
 * \description Using XDR for serialization
 */

#include <stdio.h>
#include "system_load.h"

int main(int argc, char *argv[] ) {
  XDR xdrs;
  struct SystemLoad S;
  S.system_name = NULL;
  xdrstdio_create( &xdrs, stdin, XDR_DECODE );
  SystemLoad_XDR( &xdrs, &S );
  printf("\n Name = %s : %f %f %f\n", S.system_name,
   S.avg1, S.avg5, S.avg15 );
```

```
  return( 0 );
}
```

Listing 9.10 Client RPC file for system load

```
$./rpc_info_writer | ./rpc_info_reader

Name = celex : 1.380000 1.300000 0.780000
```

When using `xdr_string` we have to initialize the memory pointed to by the structure, to NULL, or a valid memory location (which can atleast be of size MAXLEN). If the memory is NULL, XDR will call `malloc`; this memory should be freed afterwards. Another example of XDR, this time using it in conjunction with `zlib` is given in Listing 8.1.

9.2 Checksum computation

Alongwith data transmission and storage, comes the possibility of data corruption. Data can be corrupted due to hardware or software failures, and reliable communication channels thus always include a *checksum* with each packet of data. The capability of the checksum depends on the expected rate of failure, system requirements on reliability, and performance. Cyclic redundancy check (CRC) is a common checksum function and has been optimized for implementation in hardware. Boost includes a CRC function, and we describe its use by an example as shown in Listing 9.11.

```
// \file crc_example.cpp
// \author Sandeep Koranne, (C) 2010
// \description CRC example using Boost
#include <iostream>          // for program IO
#include <fstream>           // for data IO
#include <boost/crc.hpp>     // actual CRC
#include <string>            // for filename

int CalculateCRC32( std::ifstream& ifs ) {
  ifs.seekg( 0, std::ios::beg ); // seek to begining
  boost::crc_32_type boost_crc;
  unsigned long file_size = 0;
  while( ifs ) {
    unsigned char uc = ifs.get();
    boost_crc.process_byte( uc );
    file_size++;
  }
  std::cout << "File checksum = " << boost_crc.checksum()
            << " processed " << file_size << " bytes.\n";
}

int main( int argc, char * argv[] ) {
  if( argc != 2 ) { std::cerr << "Usge: crc32 <filename>\n";
    exit( 1 );
  }
  std::string fileName( argv[1] );
  std::ifstream ifs( fileName.c_str() );
  if( !ifs ) {
```

```
    std::cerr << "Unable to open file: " << fileName << " for reading.\n";
    exit( 1 );
  }
  CalculateCRC32( ifs );
  return 0;
}
```

Listing 9.11 Example of Boost CRC32 checksum

In addition to CRC32, boost provides the following checksum algorithms:

1. `crc_16_type`
2. `crc_ccitt_type`
3. `crc_xmodem_type`
4. `crc_32_type`

9.2.1 MD5

Simple CRC32 (as discussed above in Boost CRC) is prone to *aliasing*. Using 128-bit MD5 (RFC 1321) can alleviate this concern to a great extent. Although the GNU/Linux command `md5sum` can compute the MD5 checksum of a given file, we can also use MD5 checksum on arbitrary frames of data (e.g., when devising an error correcting protocol, or disk file system). Using APR MD5 functionality we can quickly integrate MD5 checksum into an application. See the small example listing shown in Listing 9.12.

```
// \file md5example.cpp
// \author Sandeep Koranne (C) 2010
// \description Example of using MD5 checksum
#include <iostream>           // for program IO
#include <iomanip>            // for std::hex
#include <cstdio>             // for C FILE*
#include <cstdlib>            // for exit
#include <apr_general.h>      // APR general
#include <apr_mmap.h>         // APR mmap
#include <apr_md5.h>          // MD5 computation

static const int MAX_NUMBER = APR_MD5_DIGESTSIZE;
static void ComputeMD5Checksum( const char *filename,
        unsigned char digest[MAX_NUMBER]) {
  apr_status_t retval;
  apr_pool_t *pool;
  apr_pool_create( &pool, NULL );
  apr_mmap_t *mmap;
  apr_finfo_t file_info;
  apr_file_t  *fp;
  retval = apr_file_open( &fp, filename,
        APR_READ|APR_BINARY,
        APR_OS_DEFAULT, pool );

  retval = apr_file_info_get( &file_info, APR_FINFO_SIZE, fp );
  std::cout << "File size = " << file_info.size << std::endl;
  // do the actual memory mapping
  retval = apr_mmap_create( &mmap, fp, 0, file_info.size,
          APR_MMAP_READ, pool );
  if( retval != APR_SUCCESS ) {
    std::cerr << "APR mmap failed..\n";
```

```
    exit( 1 );
  }
  apr_file_close( fp ); // file can be closed now
  // now mmap->mm is a const char* which can be read
  apr_md5_ctx_t md5context;
  apr_md5_init( &md5context );
  apr_md5( digest, mmap->mm, file_info.size );
  apr_mmap_delete( mmap ); // return back memory
  apr_pool_destroy( pool );
}


int main( int argc, char *argv[] ) {
  // Step 1. initialize APR
  if( argc != 2 ) {
    std::cerr << "Usage: ./md5example <filename>\n";
    exit(1);
  }
  const char *filename = argv[1];
  apr_initialize();
  unsigned char digest[ MAX_NUMBER ];
  ComputeMD5Checksum( filename, digest );
  std::cout << "MD5 digest for " << filename << " = \n";
  for( int i=0; i < MAX_NUMBER; ++i )
    std::cout << std::hex << (int)digest[i];
  std::cout << std::endl;
  apr_terminate();
  return (0);
}
```

Listing 9.12 Example of computing MD5 checkum using APR

Compiling and running this program on the binary itself gives us:

```
$md5sum -b md5example.cpp
a7af280c4d7c627b97f080a98074ecf5 *md5example.cpp
$ ./md5example md5example
File size = 9723
MD5 digest for md5example =
a0cf97256ae47ba16df66eee1fe42c3
```

We first run the GNU/Linux program with the '-b' option (for binary files), thereafter, we run our own program which uses `mmap` on the given file.

9.2.2 SHA1 checksum

Simple CRC32 (as discussed above in Boost CRC) is prone to *aliasing*. Using SHA1 (NIST Secure Hash Algorithm) can alleviate this concern to a great extent. Although the GNU/Linux command `sha1sum` can compute the SHA1 checksum of a given file, we can also use SHA1 checksum on arbitrary frames of data (e.g., when devising an error correcting protocol, or disk file system). Using APR SHA1 functionality we can quickly integrate SHA1 checksum into an application. See the small example listing shown in Listing 9.13.

```
// \file shalexample.cpp
```

```
// \author Sandeep Koranne (C) 2010
// \description Example of using MD5 checksum
#include <iostream>           // for program IO
#include <iomanip>            // for std::hex
#include <cstdio>             // for C FILE*
#include <cstdlib>            // for exit
#include <apr_general.h>      // APR general
#include <apr_mmap.h>         // APR mmap
#include <apr_sha1.h>         // SHA1 computation

static const int MAX_NUMBER = APR_SHA1_DIGESTSIZE;
static void ComputeSHA1Checksum( const char *filename,
                                 unsigned char digest[MAX_NUMBER]) {
  apr_status_t retval;
  apr_pool_t *pool;
  apr_pool_create( &pool, NULL );
  apr_mmap_t *mmap;
  apr_finfo_t file_info;
  apr_file_t  *fp;
  retval = apr_file_open( &fp, filename,
                          APR_READ|APR_BINARY,
                          APR_OS_DEFAULT, pool );

  retval = apr_file_info_get( &file_info, APR_FINFO_SIZE, fp );
  std::cout << "File size = " << file_info.size << std::endl;
  // do the actual memory mapping
  retval = apr_mmap_create( &mmap, fp, 0, file_info.size,
                            APR_MMAP_READ, pool );
  if( retval != APR_SUCCESS ) {
    std::cerr << "APR mmap failed..\n";
    exit( 1 );
  }
  apr_file_close( fp ); // file can be closed now
  // now mmap->mm is a const char* which can be read
  apr_sha1_ctx_t sha1context;
  apr_sha1_init( &sha1context );
  apr_sha1_update_binary( &sha1context, (unsigned char*)(mmap->mm),
        file_info.size );
  apr_sha1_final( digest, &sha1context );
  apr_mmap_delete( mmap ); // return back memory
  apr_pool_destroy( pool );
}


int main( int argc, char *argv[] ) {
  // Step 1. initialize APR
  if( argc != 2 ) {
    std::cerr << "Usage: ./sha1example <filename>\n";
    exit(1);
  }
  const char *filename = argv[1];
  apr_initialize();
  unsigned char digest[ MAX_NUMBER ];
  ComputeSHA1Checksum( filename, digest );
  std::cout << "SHA1 digest for " << filename << " = \n";
  for( int i=0; i < MAX_NUMBER; ++i )
    std::cout << std::hex << (int)digest[i];
  std::cout << std::endl;
  apr_terminate();
  return (0);
}
```

Listing 9.13 Example of computing SHA1 checkum using APR

Compiling and running this program on the binary itself gives us:

```
$ sha1sum sha1example
```

```
9ca4b2a1fb03d3038c22e313a1cebb8a39069400  sha1example
./sha1example sha1example
File size = 9872
SHA1 digest for sha1example =
9ca4b2a1fb3d338c22e313a1cebb8a396940
```

9.3 OpenSSL

The OpenSSL Project is an open-source effort to develop a robust toolkit implementing the Secure Socket Layer (SSL v2/v3) and Transport Layer Security (TLS v1) as well as a general purpose cryptographic library. The `openssl` command-line tool can be used for:

1. Creation and management of public/private keys,
2. Public key cryptography,
3. Creation of X.509 certificate,
4. Calculation of message digests,
5. Encryption and decryption with Ciphers.

`openssl` command-line tool is a general purpose utility which can invoke the processing functions listed above. For example, the MD5 digest can be computed using:

```
$openssl version
OpenSSL 1.0.0-fips-beta4 10 Nov 2009
$openssl md5 math.lisp
MD5(/home/skoranne/math.lisp) =
   d82e9c3808c986bce26df25640c80886
```

Some of the other tools in OpenSSL can be listed using `openssl -h`

```
Standard commands
asn1parse  ca         ciphers   cms
crl        crl2pkcs7  dgst      dh
dhparam    dsa        dsaparam  enc
engine     errstr     gendh     gendsa
genpkey    genrsa     nseq      ocsp
passwd     pkcs12     pkcs7     pkcs8
pkey       pkeyparam  pkeyutl   prime
rand       req        rsa       rsautl
s_client   s_server   s_time    sess_id
smime      speed      spkac     ts
verify     version    x509

Message Digest commands
md2        md4        md5       rmd160
sha       sha1

Cipher commands
```

```
aes-128-cbc aes-128-ecb   aes-192-cbc  aes-192-ecb
aes-256-cbc aes-256-ecb   base64       bf
bf-cbc      bf-cfb        bf-ecb       bf-ofb
camellia    camellia-256  cast         cast-cbc
cast5-cbc   cast5-cfb     cast5-ecb    cast5-ofb
des         des-cbc       des-cfb      des-ecb
des-ede     des-ede-cbc   des-ede-cfb  des-ede-ofb
des-ede3    des-ede3-cbc  des-ede3-cfb des-ede3-ofb
des-ofb     des3          desx         rc2
rc2-40-cbc  rc2-64-cbc    rc2-cbc      rc2-cfb
rc2-ecb     rc2-ofb       rc4          rc4-40
seed        seed-cbc      seed-cfb     seed-ecb
seed-ofb    zlib
```

9.4 XML Processing

XML (Extended Markup Language) is a standard for representing markup languages. SVG files, configuration files for GNOME, and many other systems now use XML to communicate and store data. For application programs to read and write XML files, they can use software API. Some libraries for XML handling are:

1. SAX: SAX (Simple API for XML) is a standard interface for event-based XML parsing,
2. libXML: a SAX (Simple API for XML) like parser,
3. Expat: another event-based parsing framework,
4. APR XML API: Apache Portable Runtime (APR) has XML processing capabilities,
5. Boost XML: Boost C++ API also has XML handling.

Consider an example XML file:

```
<settings>
  <profiles>
    <profile>
      <id>JPP</id>
        <repositories>
          <repository>
            <id>internal</id>
            <layout>jpp</layout>
            <url>file:///builddir/build/BUILD/\
                   jetty-6.1.20/.m2/repository</url>
          </repository>
          <repository>
            <id>external</id>
            <layout>jpp</layout>
            <url>file:///builddir/build/BUILD/\
                   jetty-6.1.20/external_repo</url>
...
```

9.4.1 Expat : XML processing

An example of using Expat for XML parsing is shown in Listing 9.14.

```
static void XMLDeclaration( void *ud, // user data
          const XML_Char *version,
          const XML_Char *encoding,
          int standalone ) {
  std::cout << "XML Declaration : Version = " << version << "\n"
            << "Encoding = " << encoding << "\n";
}


static void ParseElementDecl( void *ud,
            const XML_Char *name,
            XML_Content *model ) {
  std::cout << "Read ElementDecl : " << name << "\n";
}

static void EntityDecl( void *ud, // user data
      const XML_Char *name,
      int is_parameter,
      const XML_Char *value,
      int value_length,
      const XML_Char *base,
      const XML_Char *systemId,
      const XML_Char *publicId,
      const XML_Char *notationName ) {
  std::cout << "Entity : " << name << "\n";
}

static void StartDocHandler( void *ud,
           const XML_Char *doctypeName,
           const XML_Char *sysid,
           const XML_Char *pubid,
           int has_internal_subset ) {
  std::cout << "Doc type name = " << doctypeName << "\n";
}

static void DefaultHandler( void *ud,
          const XML_Char *s,
          int len ) {
  std::cout << "Default : " << s << "\n";
}


static void ProcessXMLStream( const char* data, size_t len ) {
  XML_Parser parser = XML_ParserCreate( NULL );
  XML_SetElementDeclHandler( parser, ParseElementDecl );
  XML_SetXmlDeclHandler( parser, XMLDeclaration );
  XML_SetEntityDeclHandler( parser, EntityDecl );
  XML_SetStartDoctypeDeclHandler( parser, StartDocHandler );
  XML_SetDefaultHandler( parser, DefaultHandler );
  XML_Parse( parser, data, len, 1 ); // is final
}
```

Listing 9.14 Example of using Expat for XML parsing

9.4.2 libXML : XML processing library

Another API for XML processing is the `libxml` library, which can be configured to read/write XML files using event-processing, or data-driven mode. An example of the data-driven mode is shown in Listing 9.15.

```
// \file libxml_example.cpp
// \author Sandeep Koranne (C) 2010
// \description XML parsing using libxml
#include <cstdio>             // for FILE
#include <iostream>           // for program IO
#include <libxml/parser.h>    // libxml

static void ProcessFile( const char *filename ) {
  xmlDocPtr doc;
  xmlNodePtr cur;
  doc = xmlParseFile( filename );
  if( doc == NULL ) {
    std::cerr << "Unable to parse XML file : " << filename << "\n";
    exit(1);
  }
  cur = xmlDocGetRootElement( doc );
  std::cout << "Root = " << cur->name << "\n";
  while( cur ) {
    std::cout << cur->name << "\n";
    xmlNode *children = cur->children;
    while( children ) {
      std::cout << "\t Child = " << children->name << "\n";
      children = children->next;
    }
    cur = cur->next;
  }
  xmlFreeDoc( doc );
}

int main( int argc, char *argv [] ) {
  if( argc != 2 ) {
    std::cerr << "Usage: ./libxml_example <file>.xml...\n";
    exit(1);
  }
  ProcessFile( argv[1] );

  std::cout << std::endl;
  return (0);
}
```

Listing 9.15 Example of using libxml for XML processing

The code in Listing 9.15 should be contrasted with Listing 9.14; in Listing 9.14 we defined *callbacks* which the API called for us on specific events during the parsing of the XML file. In Listing 9.15 we explore the XML data-structure tree using our own control loops. Compiling the code in Listing 9.15 and running it on the example XML file shown above gives:

```
Root = settings
settings
 Child = text
 Child = profiles
 Child = text
 Child = activeProfiles
```

```
 Child = text
.....
```

XML handling is also present in other languages such as Perl and Python, which often provide *bindings* to one or more of the above libraries, thus, the syntax and method of using XML libraries is as described above.

9.5 Berkeley DB

Berkeley DB is an open-source embedded database library that provides scalable, high-performance, transaction protected data management services to applications. Berkeley DB provides a simple API for data-access and management. Bindings for Berkeley DB are provided for C, C++, Perl, Java, Tcl, Python, Lisp and many other languages. Berkeley DB runs in the same memory space as the application, and as such no inter-process or network communication is required to use Berkeley DB.

Berkeley DB supports many different storage structures including: (i) hash tables, (ii) BTrees, (iii) simple record-number based storage,dow and (iv) persistent queues. Programmers can create tables using these structure. Hash tables are best suited for large applications with a need for predictable search time, while BTrees are better for range-based searches.

Berkeley DB also provides data management services such as concurrency, transactions and recovery. Records in Berkeley DB are represented as *(key,value)* pair, and it provides functions for:

1. Insertion: insert a record in a table,
2. Search: find a record in a table,
3. Deletion: delete a record from a table,
4. Update: update a record in a table.

Berkeley DB is not a relational database and does not support SQL. It is possible to build a relational database using the transactional library and data storage facilities of Berkeley DB (infact, MySQL used Berkeley DB for its storage).For SQL oriented databases refer to Chapter 20 for a discussion on open-source relational databases. Keys and values can be arbitrary bit strings (fixed-length or variable length, depending on the application).

9.5.1 DB open function

The `DB->open()` function opens the database for writing as well as reading. The form of this function is as follows:

```
#include <db.h>
int DB->open( DB *db, DB_TXN *txnid, const char *file,
              const char *database, DBTYPE type,
```

```
                u_int32_t flags, int mode);
```

The DB is the *handle* to the database, and `txnid` is the transaction id if transactions are used, else NULL. The file name of the database is passed in `file`. The `database` argument is optional; it allows for the application to have several databases in the same logical file. If there is a single database, this argument can be left NULL. The `DBTYPE type` specifies the type of the database, whether DB_BTREE, DB_HASH, DB_QUEUE, DB_RECNO, or DB_UNKNOWN. The `flags` specify whether the database should be created (DB_CREATE), or opened for read-only access (DB_RDONLY), or whether the database should be multiversioned (DB_MULTIVERSION). The `mode` parameter is used to set the file permission on the created file.

A database handle is created using `db_create` and subsequently a database can be opened using `DB->open()` function. When the processing is complete the database should be closed using the `DB->close()` function. Consider an example application written using the Berkeley DB as shown in Listing 9.16.

```
// \file bdb_example.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of using Berkeley DB

#include <cassert>            // for assertion checking
#include <cstdio>             // C stdio
#include <cstdlib>            // C stdlib for exit, atoi
#include <cstring>            // memset
#include <iostream>           // program IO
#include <string>             // std::string
#include <sys/types.h>        // C standard types
#include <db.h>               // Berkeley db


int main( int argc, char *argv[] ) {
  if( argc != 3 ) {
    std::cerr << "Usage: bdb_example <db-name> <op-code>\n";
    exit(1);
  }
  std::string dbName( argv[1] );
  int db_operation = atoi( argv[2] );
  DB *dbp = NULL;
  int rc = db_create( &dbp, NULL, 0 );
  if( rc != 0 ) {
    std::cerr << "db_create " << db_strerror( rc );
    std::exit(1);
  }

  if( db_operation == 1 ) { // database create
    rc = dbp->open( dbp, NULL, dbName.c_str(),
                    NULL, DB_BTREE, DB_CREATE, 0664 );
    if( rc != 0 ) {
      perror("Berkeley db open error");
      exit(1);
    }
    // successfully opened the db
    DBT key, data;
    memset( &key, 0, sizeof( DBT ) );
    memset( &data, 0, sizeof( DBT ) );
    key.data = (void*)"subject_A";
    key.size = sizeof( "subject_A" );
    data.data = (void*)"Math";
    data.size = sizeof( "Math" );
```

```
    rc = dbp->put( dbp, NULL, &key, &data, 0 );
    switch( rc ) {
    case 0 : { break; /* key added */ }
    case DB_KEYEXIST:{
      std::cerr << "already stored "<<key.data<< "\n"; break;
      }
    }
    key.data = (void*) "subject_B";
    data.data = (void*) "Physics";
    data.size = sizeof( "Physics" );
    rc = dbp->put( dbp, NULL, &key, &data, 0 );
    dbp->close( dbp, 0 );
  } else if( db_operation == 2 ) { // database open and print
    rc = dbp->open( dbp, NULL, dbName.c_str(),
                    NULL, DB_BTREE, DB_RDONLY, 0664 );
    if( rc != 0 ) {
      perror("Berkeley db open error");
      exit(1);
    }
    DBT key, data;
    memset( &key, 0, sizeof( DBT ) );
    memset( &data, 0, sizeof( DBT ) );
    key.data = (void*)"subject_A";
    key.size = sizeof( "subject_A" );
    rc = dbp->get( dbp, NULL, &key, &data, 0 );
    if( rc == 0 ) {
      std::cout << (char*)key.data << "\t" << (char*)data.data << "\n";
    } else {
      perror(" db->get error...\n");
      exit(1);
    }
    dbp->close( dbp, 0 );
  } else {
    std::cerr << "Unknown db-operation, " << db_operation << std::endl;
  }

  std::cout << "Example of Berkeley db\n";

  return (0);
}
```

Listing 9.16 Example of using Berkeley DB

The functions `DB->put()` and `DB->get()` are described below:

```
#include <db.h>
int
DB->put( DB *db, DB_TXN *txnid, DBT *key, DBT *data,
         u_int32_t flags);
int
DB->get( DB *db, DB_TXN *txnid, DBT *key, DBT *data,
         u_int32_t flags);
```

The `DB->put()` function stores key/data value pairs into the database. It can either replace an existing key/value pair, or add duplicate (if allowed).

9.5.2 Other Berkeley DB functions

In addition to these above functions, Berkeley DB has a number of useful function as listed in Table 9.2.

Table 9.2 Berkeley DB functions

Database functions	Description
`DB->compact()`	compact a database
`DB->del()`	delete items from database
`DB->err()`	retrieve error message
`DB->exists()`	return if an item exists in the database
`DB->fd()`	return underlying file descriptor
`DB->get()`	get key/value pair (see above)
`DB->get_byteswapped()`	check if db is in host order
`DB->get_dbname()`	return file and db name
`DB->get_type()`	return database type
`DB->join()`	perform db join on cursors
`DB->key_range()`	return estimate on key location
`DB->open()`	open database (see above)
`DB->put()`	put key/value pair in db (see above)
`DB->remove()`	remove a database
`DB->rename()`	rename a database
`DB->stat()`	return database statistics
`DB->sync()`	flush database to disk
`DB->verify()`	verify database integrity
`DB->cursor()`	create a database cursor
BTree / Recno config	Description
`DB->set_bt_compare()`	set BTree comparison function
`DB->set_bt_compress()`	set BTree compression function

Berkeley DB also has a simple Python interface, an example of which is shown in Listing 9.17.

```
#!/usr/bin/python
#example file for using Berkeley DB with Python
import bsddb
db = bsddb.btopen('example.db', 'c')
print db.keys()
for k,v in db.iteritems():
    print k,v
```

Listing 9.17 Using Berkeley DB with Python

Running the C++ binary, followed by the Python program gives:

```
./bdb_example example.db 1
Example of Berkeley db
[skoranne@celex BDB]$ python bsdb.py
['subject_A\x00', 'subject_B\x00']
subject_A Math
subject_B Physics
```

Berkeley DB has a number of other features such as *cursors* and database *join* operations which can be used by the application software, but even by the above

functions we can see that Berkeley DB provides an elegant and effective data storage mechanism. It is thus no surprise that a number of open-source applications (such as Subversion and MySQL) use Berkeley DB for their data storage and database management needs.

9.6 Memcached Library

Memcached is a high-performance distributed memory object caching system. It is designed to reduce database load in web applications and speed up dynamic web content generation. `libmemcached` is a client library for the *memcached* protocol. Memcache implements an in-memory key-value store for small chunks of arbitrary data. This data can be the result of applications running on the server, and the results are *cached*, so that subsequent calls to the same procedure by any application running on the server can simply return the pre-computed result, thus increasing performance.

Memcached comprises of the following components:

1. Server software: stores arbitrary key/value pairs in an internal hash table,
2. Server algorithms: for cache management and purging,
3. Client software: given a list of memcached servers,
4. Client hashing algorithm: chooses server based on key, for load balancing.

Running `memcached` on a server is a simple matter of invoking the `memcached` binary. The command accepts a number of command line options:

```
-p : port number
-s : UNIX socket path (disables network support)
-m : maximum memory (default is 64 MB)
-d : run memcache as a daemon
-M : disable automatic removal of objects from cache
-t : specify number of threads (default 4)
```

The status of a running `memcached` server can be checked using the `nc` tool. By sending the message “stats settings” to the specified port of the `memcached` server we can query its disposition, as shown below:

```
$echo "stats settings" | nc localhost 11211
STAT maxbytes 67108864
STAT maxconns 1024
STAT tcpport 11211
STAT udpport 11211
STAT inter NULL
STAT verbosity 0
STAT oldest 0
STAT evictions on
STAT domain_socket NULL
STAT umask 700
STAT growth_factor 1.25
```

```
STAT chunk_size 48
STAT num_threads 4
STAT stat_key_prefix :
STAT detail_enabled no
STAT reqs_per_event 20
STAT cas_enabled yes
STAT tcp_backlog 1024
STAT binding_protocol auto-negotiate
STAT auth_enabled_sasl no
STAT item_size_max 1048576
END
```

To use the `memcached` server as a cache for key/value pair, the client software must implement the `memcached` protocol. The protocol specifies a key, flag value, an expiration time, and arbitrary data. The `memcached` protocol can be implemented in the client, an example with Python client is shown in Listing 9.18.

```
#!/usr/bin/python
# example of using memcached server
import pylibmc
mc = pylibmc.Client(["localhost"], binary = True )
mc.behaviors = {"tcp_nodelay" : True }

mc.set("Subject_A", "Math")
mc.set("Subject_B", "Physics")
mc.set("Subject_C", "Chemistry")

mc.get("Subject_A")
```

Listing 9.18 Example of memcached protocol in Python

`memcached` can help in improving the performance of an application when the application has the characteristics of being compute bound in calculating results which have also been previously computed at some point in the past. The cache replacement logic, as well as the distributed aspects of the data management can be handled by `memcached`.

9.7 SWIG interface generator

Simplified Wrapper and Interface Generator (SWIG) is an open source tool to generate bindings for programs written in C/C++ to other languages (usually scripting languages) such as Python, Tcl, Perl, and Lua.

Using SWIG is deliberately simple; by adding a small number of *directives* the C/C++ header file a complete module of code can be converted into appropriate form by SWIG. The tool generates source code which enables the *calling* of the C/C++ code in the target language. SWIG has enabled the conversion of many utility libraries written in C/C++ for use in Lisp (for example).

Consider the example of generating bindings for Python as shown below:

```
int factorial( int n ) {
  if ( n < 2 ) return 1;
```

```
  else return n * factorial(n-1);
}
```

Listing 9.19 Example of Python binding generation

We write a SWIG wrapper `factorial.i` as:

```
/* SWIG file for math */
%module math
%{
#define SWIG_FILE_WITH_INIT
#include "factorial.h"
%}
%include "factorial.h"
```

Listing 9.20 SWIG interface file for Python binding

The we run SWIG as:

```
$swig -python factorial.i
$gcc -O2 -c factorial.c
$gcc -O2 -c -I/usr/include/python2.6 factorial_wrap.c
$gcc -shared factorial.o factorial_wrap.o -o _math.so
$python
Python 2.6.2 (r262:71600, Aug 21 2009, 12:22:21)
[GCC 4.4.1 20090818 (Red Hat 4.4.1-6)] on linux2
Type "help", "copyright", "credits" ...
>>> import math
>>> math.factorial(10)
3628800
```

A more detailed example using C++ is shown below:

```
#ifndef _layer_db_h_
#define _layer_db_h_
struct Box {
  int x1,y1,x2,y2;
};

struct LayeredBox {
  int layer;
  int n;
  Box *D;
};

LayeredBox ReadData( const char* );
void PrintLayerData( const LayeredBox& B );
#endif
```

Listing 9.21 SWIG Python integration with C++ header file

```
// simple db format library
#include <stdio.h>
#include <stdlib.h>

#include "db.h"

LayeredBox ReadData( const char* fileName ) {
  int i;
```

```
  LayeredBox retval;
  FILE *fp = fopen( fileName, "rb" );
  if( !fp ) exit(1);
  fscanf( fp, "%d %d", &retval.layer, &retval.n );
  retval.D = (Box*) malloc( retval.n * sizeof( Box ) );
  for(i=0; i < retval.n; ++i) {
    fscanf( fp, "%d %d %d %d",
      &retval.D[i].x1, &retval.D[i].y1,
      &retval.D[i].x2, &retval.D[i].y2 );
  }
  return retval;
}

void PrintLayerData( const LayeredBox& B ) {
  fprintf( stdout, "Layer = %d", B.layer );
  for( int i=0; i < B.n; ++i ) {
    fprintf( stdout, "\n %d %d %d %d",
       B.D[i].x1, B.D[i].y1,
       B.D[i].x2, B.D[i].y2 );
  }
}
```

Listing 9.22 SWIG Python integration with C++ file

```
/* SWIG file for math */
%module layerdb
%{
#define SWIG_FILE_WITH_INIT
#include "db.h"
%}
%include "db.h"
```

Listing 9.23 SWIG interface file for Python binding

```
$swig -python -c++ db.i
$g++ -shared db.o db_wrap.o -o _layerdb.so
$python
Python 2.6.2 (r262:71600, Aug 21 2009, 12:22:21)
[GCC 4.4.1 20090818 (Red Hat 4.4.1-6)] on linux2
Type "help", "copyright", "credits" ...
>>> import layerdb
>>> D = layerdb.ReadData("a.data")
>>> layerdb.PrintLayerData( D )
Layer = 46
 0 0 10 10
 20 10 30 20
 5 5 10 5>>>
```

As the above example shows, SWIG and Python can maintain the object structure of complex C++ classes and structure, and pass them to and from functions.

9.8 Conclusion

In this chapter we have presented application development libraries which aid in the development of remote procedure call enabled applications. The libraries presented included RPC (remote procedure call), XDR (extensible data representation), checksum, SHA1, MD5, OpenSSL. The use of XML files for data representation was presented with examples using C++. Persistent storage of data using BerkeleyDB, and its use in C++ and Python code, was also discussed. Lastly, we presented the Simplified Wrapper Interface Generator (SWIG) tool which can automate the wrapping of C and C++ APIs for use in Python, Scheme, and other interactive languages.